

Project Report: CAPTCHA Character Localization

ECEN 640 Digital Image Processing, Fall 2025
Luca Chierotti and Nicholas Kenny
10 December 2025

1. Implementation Description

1.1 Restated Project Objectives

For this project, we decided to use the CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) dataset on Kaggle. These images do not originate from Kaggle but were instead built in 1997 as a way to block online bots. CAPTCHA images are used to verify that website users are in fact human and not online bots. They are prompted to a user such that the human has to ascertain what letters or numbers are in the given image. Such images contain occlusions such as dashed lines, blurred writing, or even varying character spacing. This is all to make them “unbreakable” to bots yet easily legible to human beings. However, due to advancements in AI, this original CAPTCHA image set has since then been replaced by reCAPTCHA.

As stated the images are gathered from the kaggle dataset, which contains at a total of 1070 samples ([images](#)). As seen in the two samples below, the images are 5 letter words that can contain any combination of numbers (0-9) and/or lower case letters (a-z). The images have had random noise applied to them: each image has a blur and a random line of varying stroke width applied. They are all grey scale 200 x 50 PNGs. As seen in the two examples, the image text is purposefully smudged and curved. The text/numbers can be slightly shifted or even have the characters touching each other. Additionally, the placement of the strokewidth line is random, never repeated, and consistently overlaps with the characters. Finally, each image also has a gradient from grey to white.



Although this dataset is intended for machine learning (ML) tasks where large datasets are useful for training models for classification, for the purposes of this project we will not use the full dataset. Instead we will use only the first 60 images of the dataset for image processing related tasks.

In each image, the number of target objects (characters) is exactly five, but their placement, spacing, orientation, stroke width, and occlusions vary significantly from image to image.

Our primary objective is to localize the five characters in each CAPTCHA by identifying their bounding boxes using image processing techniques. We will not perform character recognition as this is related to ML or deep learning. Instead, the focus is solely on segmentation and bounding-box detection.

1.2 Proposed Localization Approaches

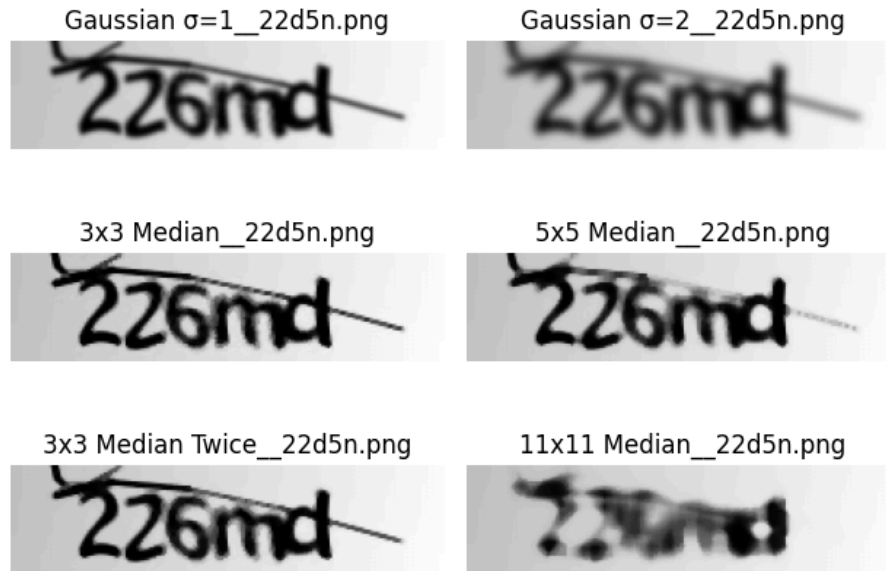
For this project, we implemented two independent image-processing approaches for character localization in the CAPTCHA dataset.

1.2.1 Approach 1: Projection Profile Segmentation

Our first approach is a purely classical image-processing pipeline that uses adaptive thresholding and horizontal/vertical projection profiles to localize the five characters in each CAPTCHA image. The method assumes that the characters occupy a reasonably compact horizontal band and produce consistent peaks in pixel density along the horizontal and vertical directions. This approach proceeds in 3 main stages: horizontal projection and band selection, vertical projection and cut detection, and finally bounding-box extraction with post-filtering.

Horizontal Projection (Y-Segmentation)

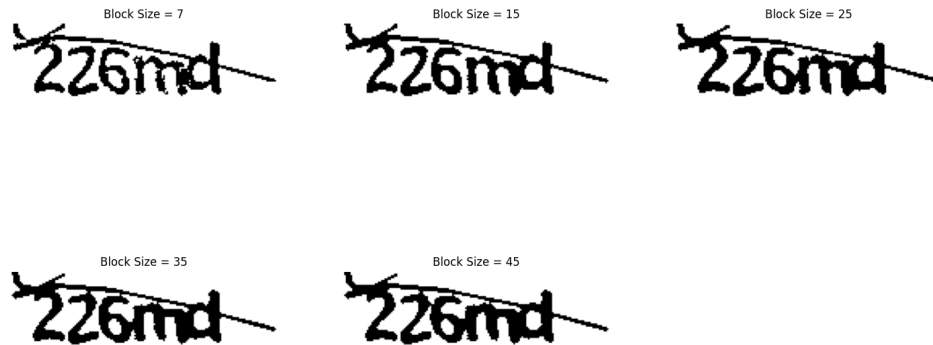
The first step in the horizontal projection process is the preprocessing stage. This is meant to obtain a clean binary representation of each CAPTCHA image. Firstly, the greyscale input image is fed through a smoothing filter such that the character edges are easier to discern. Such a filter “smooths” out the “salt and pepper” noise seen in the images of the page above (e.g. ‘6’ and ‘m’ characters in the first example image). The figure below shows the results of testing various filter types and sizes.



The figure shows how the 3x3 median is good at preserving edges while removing noise specks (salt-and-pepper noise). It handles the fuzzy noise inherent in the CAPTCHA better than the gaussian filter for edge preservation. Thus, the 3x3 median filter was chosen.

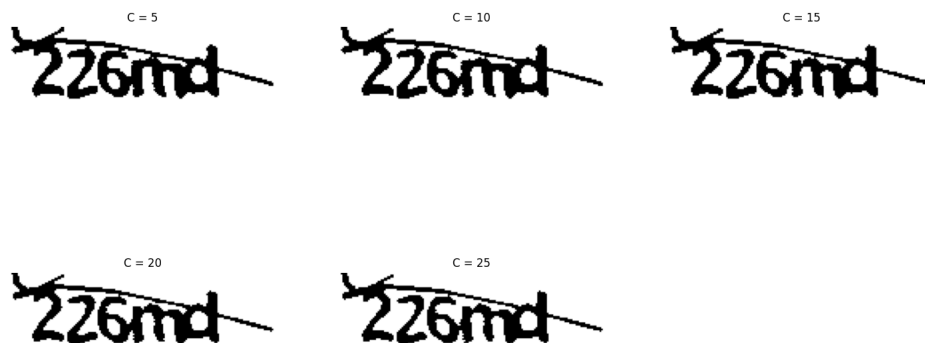
On this smoothed image, local mean adaptive thresholding is then applied. Adaptive thresholding works by calculating a unique threshold value for a small neighborhood around each pixel, rather than using one global threshold for the entire image. The general formula for an adaptive threshold is $T(x, y) = \text{Statistic}(x, y) - C$. Here the $\text{Statistic}(x, y)$ is the local mean (average) of the pixel intensities within a defined neighborhood (block size). C is a small, constant subtraction value used to bias the threshold. Since we were unsure of what block size and C value to use, we plotted them to decide which values to use. The plots below show how we first decided on a fixed block size of 15. Although the block size subplots all seem very similar, the block size = 15 gives the best balance. A smaller block size causes the image to become too grainy after binarization, while a larger size is more likely to merge characters together.

Effect of Adaptive Threshold Block Size



After the block size was set, the threshold constant was similarly chosen through visual inspection and trial and error. We can see from the below image how they are all very similar. However, the $C = 25$ plot showed, again, to be the value which merged the characters the least in the final image; it resulted in the image with the most distinct characters. Thus, this value was chosen. The final product of this section results in a fully binarized image

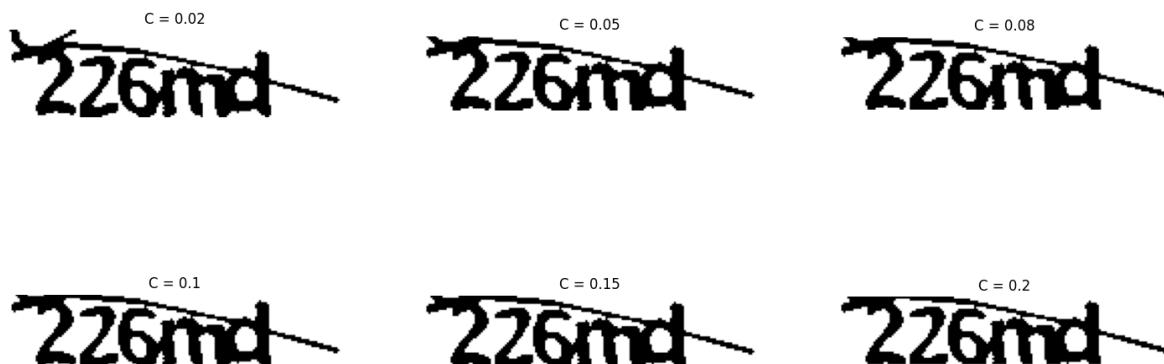
Effect of Adaptive Threshold Constant C



From this preprocessed image we want to obtain the central horizontal band where the 5 characters reside. The intention of this is to crop out the top and bottom of the stroke width line and “isolate” the main character region. This is done by creating a horizontal projection profile. This profile is calculated by summing the pixel values across each row. This results in a 1D array where each element of the projection profile represents the total number of black (character) pixels in a given row. As one can infer, a high value means a row with dense character content. Conversely, a low value means the row mostly consists of background pixels.

Once the profile is calculated, the next step is to identify the character band by thresholding the profile values. Each value is compared against a threshold. This threshold is a percentage multiplied by the image width: $\text{threshold} = \text{threshold_percentage} * \text{Width}$. In a similar way as during the image preprocessing, the value of the percentage value was chosen through visual inspection. We chose to implement a percentage threshold method as opposed to a more dynamic method, as this seemed the easiest and most time efficient to implement the horizontal projection step.

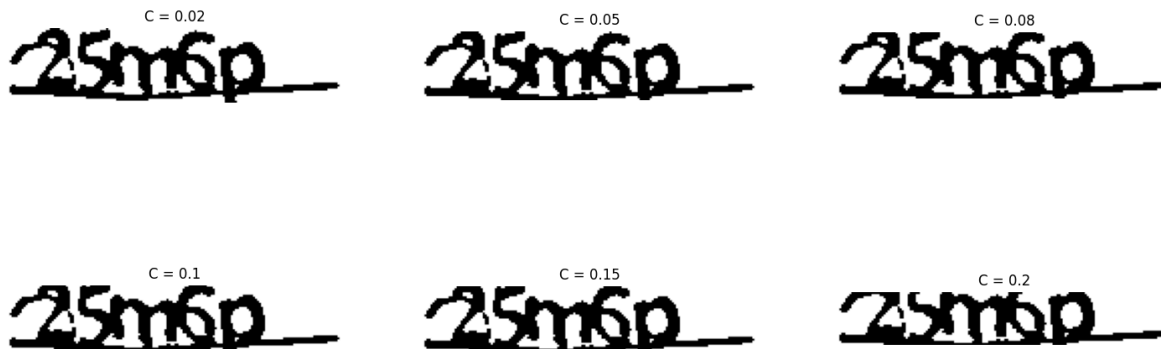
Effect of Threshold Percentage C



Paying notice to the upper left corner of each image, the plot above demonstrates how the value of the percentage threshold results in different “bands”. The low valued thresholds ($c = 0.02$), result in little to no cropping. Conversely, the high valued percentage values ($c = 0.15, 0.2$) offer similar aggressive cropping: the bottom on the characters start to get “shaved off”. Through testing, we found that the $c = 0.1$ percentage value provided the correct sweet spot which generalized to the majority of images. Although the aggressive

cropping of $c = 0.2$ or $c = 0.15$ was successful in cropping the strokewidth line for this image, these c values did not generalize to other images well as seen below.

Effect of Threshold Percentage C



This image shows for a separate sample image, that $c = 0.2$ and $c = 0.15$ are much too aggressive at cropping. As such the $c = 0.08$ was chosen as the safe middle ground, which allowed this horizontal projection method to generalize to all the images in our dataset.

Vertical Projection (X-Segmentation)

Once the horizontal projection for each image is calculated, the next step is to obtain the vertical projection (cutout) for each of the 5 characters. Instead of isolating the full vertical character band (opposite to the previous section) here intention is to isolate each character. The goal here is thus not to determine foreground vs. background pixels but instead to find the valleys (potential cut points) between touching or non-touching characters.

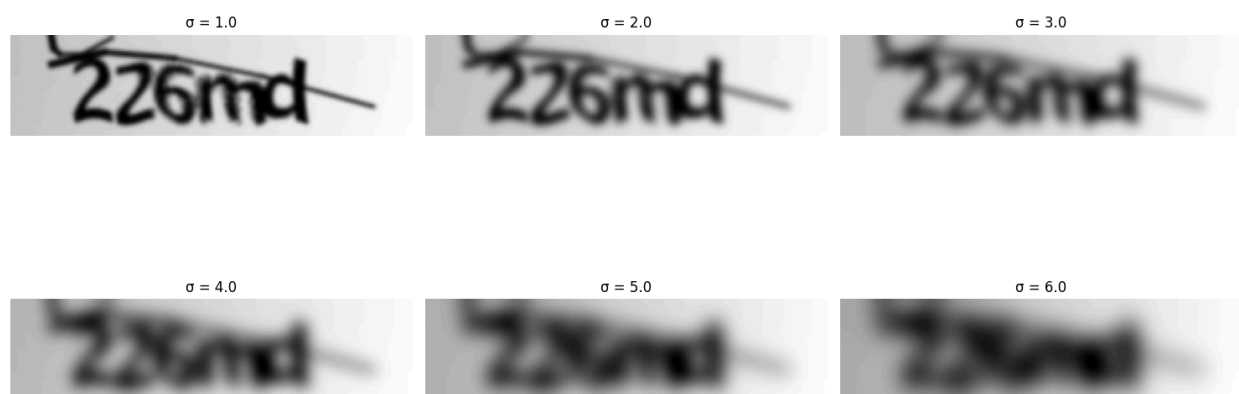
However, one might wonder, why not repeat the same thresholding method as in the horizontal projection method. As we quickly discovered, this approach does not lead to effective or accurate results. The characters have noisy edges that sometimes overlap, merging the characters. They also inherently are shaped non-uniformly. For example, the character "m" has many false peaks and minimas which would potentially cause the algorithm to determine it as multiple characters and cut it accordingly. Additionally, the strokewidth line presents another issue as it overlaps with the characters, which could

cause even more false peaks and minimas. Furthermore, in testing when we used the binary CAPTCHA image from the preprocessing step, the image's touching letters, uneven intensity, internal holes, and slanted strokewidth line, all resulted in an extremely jagged vertical projection with many artificial minima. As one can imagine, this caused very inaccurate results.

To resolve these issues we decided to try a new preprocessing approach. We decided to use the original unprocessed CAPTCHA images and feed them into a gaussian filter followed by an OTSU filter.

The first step we used was to apply a gaussian smoothing filter to the images. The purpose of such a smoothing filter was not to blur the characters beyond recognition but rather to stabilize the vertical projection. Without smoothing, the projection would oscillate sharply due to noise, stroke roughness, and tiny gaps within the characters. Using a gaussian filter helped "normalize" character strokes by softening small fluctuations/noise, merging miniature voids, and suppressing jagged edges. This resulted in a projection profile that was smoother and far easier to interpret. The sigma value controls how aggressively this smoothing happens. The figure below shows the testing results of using different sigma values.

Effect of Gaussian Smoothing (σ)

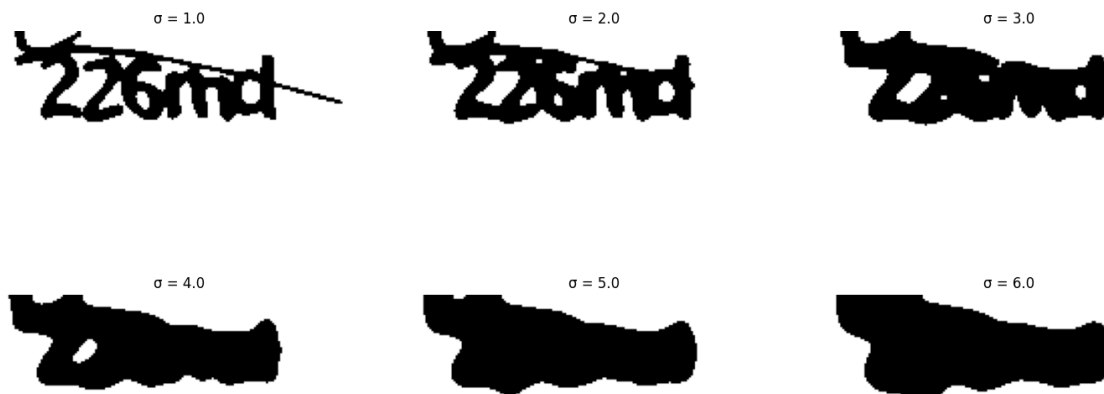


As shown in the figure above, small sigma values (1.0–2.0) preserved too much detail leaving many noisy oscillations. An extremely large sigma (5.0–6.0) oversmoothed the

image and made characters “melt together” into a single blob. A middle value (sigma = 3.0-4.0) turned out to be the most appropriate compromise: it removes the high-frequency noise that destabilizes the projection but preserves the separations between characters well enough for valley detection to work.

After smoothing, we apply Otsu’s thresholding. As we are using the openCV library for this project, `otsu` is a function inherent to the library. Otsu’s method computes an optimal global threshold by analyzing the image histogram and choosing the one single threshold value that maximizes the division between two intensity classes (characters and background). Otsu does not rely on local structure. Instead, it gives a single clean binary image that reflects the smoothed intensity transitions. Because smoothing (from the gaussian) has already reduced the character-level noise, the Otsu produces a binary version with (in theory) a stable and noise-resistant global threshold. The image below shows the effect of the various sigma values on the resulting Otsu thresholding.

Effect of Gaussian Smoothing (σ)



By feeding this smoothed + Otsu-processed image into the next vertical projection processing steps, the resulting profile becomes substantially cleaner and more stable. Thus, it makes the valleys between characters far easier to detect.

In order to make the valleys easier to detect, we first transform the projection into an “inverted” form by subtracting each column’s ink (black pixel) count from the global maximum. This inverse allows us to use the scipy library’s `flips_peaks` function. This function is normally used for detecting maxima in an image. The function here is used to

identify potential cut points as “local minima” in the original (non-inverted) projection. Thus, the `flips_peaks` function allows us to find potential “valley” indices.

Additionally, we also determine the leftmost and rightmost columns that contain any ink at all. This ensures that segmentation never attempts to cut outside the true character region.

Subsequently, the next stage is to then segment the image. The first stage of cutting uses the output of `find_peaks`. However, a problem we frequently encountered was that these outputs still included characters that touched, overlapped, or produced weak minima. Thus, this first pass often produced fewer cuts than the expected number of characters. To address this, a second, more targeted approach is applied. Here the space between each pair of existing cut points is inspected, and if a segment is too wide (larger than roughly 1.5 x the minimum character width), we look for the lowest local value in that region. If that minimum is shallow enough (below a predefined floor) it is treated as an additional valley, and a new cut is inserted. This process repeats until either enough cut points are created or no further meaningful minima can be found. The two tunable parameters used in this step, `MIN_CHAR_WIDTH` and `OPTIMAL_CUT_FLOOR`, were not arbitrarily chosen.

We examined several CAPTCHA samples side-by-side. Each character, whether thin (like “1” or “i”) or wide (like “m” or “w”), consistently occupies a horizontal span of at least 10–12 pixels after our preprocessing steps. No true character ever appears thinner than this, even under variation from smoothing, thresholding, noise, or the strokewidth line. Because of this observation, we set `MIN_CHAR_WIDTH` = 10 to encode the minimum width of a character. However, we multiply this value by 1.5 (giving 15 pixels) to ensure that only abnormally large merged-character segments are forced to split. In other words, if a segment is wider than 1.5 x the minimum character width, it is likely that two characters are stuck together and require a cut.

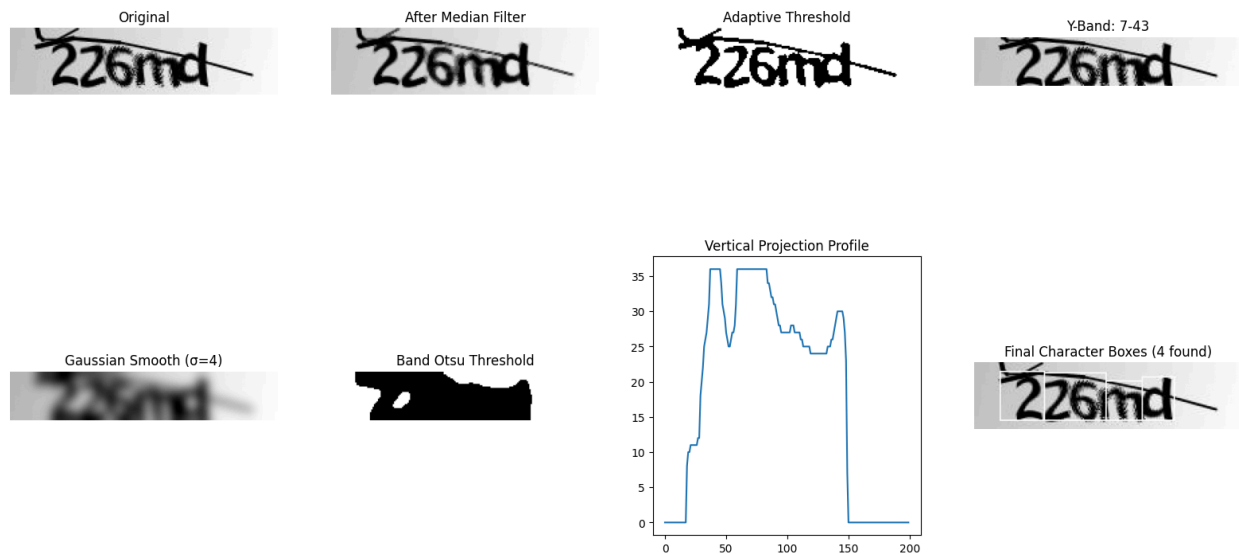
The second parameter, `OPTIMAL_CUT_FLOOR`, is derived from analyzing the vertical projection curve across many training images. In a properly smoothed and Otsu-binarized band, true valleys between characters consistently have very low column-ink counts. We found these to often be between 0 and 7 pixels depending on noise. Conversely, the interior of characters rarely dips that low; even thin characters maintain higher pixel counts. By sweeping through many images and plotting the projection values, we found that a cutoff of approximately 6 pixels reliably separates “true valleys” from “internal dips”

inside a character. Setting `OPTIMAL_CUT_FLOOR = 6` therefore prevents accidental cuts through the middle of characters while still forcing cuts in merged sections.

Bounding Box Extraction

After this, the next steps, therefore, are to iterate through adjacent cut pairs and extract the “boxed character” regions. Here we invert the inverted image character section back to normal. This is to use the opencv functions `findNonZero` and `boundingRect`. These functions find the coordinates of the character “ink” regions and compute the bounding box. Subsequently, since we commonly ran into the issue of incorrectly finding too many bounding boxes (character regions), we sorted the bounding boxes for a given image by area. We do this to select the largest 5 regions left to right. Thus, after this, the process is complete. The figure below shows all the steps listed above in chronological order.

Step-by-Step CAPTCHA Segmentation Pipeline



1.2.2 Approach 2: Parameter Auto-Tuning using IOU

Our second approach is quite similar to the first method but instead of arbitrary parameter optimization, we make use of the intersection over union (IOU) metric to perform automated optimization of parameters for the various layers of the image processing pipeline. The baseline implementation contains 5 different stages:

1. Preprocessing
2. Thresholding
3. Noise Removal
4. Segmentation
5. Post Processing

To refine these 5 different stages, we make use of 6 different categories of hyperparameters:

1. Preprocessing
 - a. Ex. Gaussian Kernel size, CLAHE tile size
2. Thresholding
 - a. Ex. Adaptive Block Size, Otsu weight
3. Morphological Operations
 - a. Ex. Kernel Size, opening and closing iterations
4. Noise Removal
 - a. Ex. Minimum Component Area, Aspect Ratio Threshold, line thickness
5. Segmentation
 - a. Ex. Character Width, Minimum Peak Width
6. Bounding Box Refinement
 - a. Ex. Box Dimensions, padding

Preprocessing

Similar to the first implementation, the first step is to convert the image to a grayscale format to make processing the image easier. The next step is to utilize the Contrast Limited Adaptive Histogram Equalization (CLAHE) algorithm for contrast enhancement of the image. CLAHE is similar to the more traditional adaptive histogram equalization method which splits the image into many tiles (typically 8x8 is the baseline) and each of these tiles is histogram equalized per usual. The additional steps which CLAHE adds to this is contrast limiting and bilinear interpolation. Before performing adaptive histogram thresholding, if any histogram bin is above the specified contrast limit (by default 40 in OpenCV), those pixels are clipped and distributed uniformly to other bins before applying histogram equalization [2]. After equalization, bilinear interpolation – which is a mathematical technique for estimating a value at a point within a 2D space, based on the

known values of four surrounding points and finding the weighted average these points that form a rectangle around the target point – is applied to remove artifacts in tile borders [3].

Following the application of CLAHE to the image, we attempt to remove the gray to white gradient common in many of the image backgrounds. To do this we setup a simple linear gradient and subtract this from the original image:

```
# Gradient compensation (removes gray-to-white gradient)
h, w = enhanced.shape
# Fit a polynomial surface to the image
y_coords, x_coords = np.mgrid[0:h, 0:w]
# Simple linear gradient removal
gradient_x = np.linspace(0, 30, w, dtype=np.float32)
gradient_2d = np.tile(gradient_x, (h, 1))

# Subtract gradient
enhanced_float = enhanced.astype(np.float32)
corrected = np.clip(enhanced_float - gradient_2d, 0, 255).astype(np.uint8)
```

The last two steps of preprocessing are bilateral filtering and adding Gaussian blur to the image. Contrary to our original understanding, bilateral filtering is different from bilateral interpolation which we discussed previously. Bilateral filtering actually makes use of two Gaussian filters, where one is a function of space to make sure that physically nearby pixels are considered for blurring, while the second is a function of intensity difference to filter through the neighboring pixels and determine which ones have similar intensities so they can be considered for blurring [4]. This distinction of having the Gaussian filter as a function of intensity is crucial because it helps to preserve edges in the image while still smoothing noise. Finally, we apply a Gaussian blur to the image to remove the last bit of Gaussian noise from the image by effectively acting like a low-pass filter.

Thresholding the Image

In this second stage, we apply adaptive thresholding to the image by combining local and global methods. The first method we use is Otsu's thresholding, similar to the first implementation. After that, we utilize adaptive thresholding to determine a pixel's threshold value based on the block of pixels around it. Specifically, we are utilizing Gaussian adaptive thresholding where the threshold value is a Gaussian-weighted sum of

the pixels in the block minus the constant C [5]. In our implementation, both the block size and C are tunable parameters.

Noise Removal

We continue the process by continuing to strip noise from the image, and with this stage we are aiming to remove the noise line that stretches through each image. To accomplish this goal, we first define the structuring element (SE) using “cv2.getStructuringElement” which actually allows us to define the shape of the structuring element [6]. We used an ellipse for our SE which is well suited to connecting the gaps in characters that form after erosion. After defining the SE, we implement morphological opening and closing using the same kernel (SE) but with variable amounts of iterations, which are also tuned parameters.

Furthermore, to specifically target the line, we used a component analysis approach. Firstly, we use “cv2.connectedComponentsWithStats” to identify all of the connected components (like lines). Then we analyze each component to determine its aspect ratio and thickness. We then filter out this noise using the following criteria:

```
# Calculate aspect ratio and thickness
aspect_ratio = max(w, h) / (min(w, h) + 1e-5)
length = max(w, h)
thickness = min(w, h)

# Filter criteria - very specific to noise lines
# Noise line characteristics:
# - Very elongated (aspect ratio > 10)
# - Spans significant portion of image (> 60% width)
# - Thin (thickness 1-4 pixels)
# - Not too small in area
is_very_elongated = aspect_ratio > self.params['aspect_ratio_threshold']
is_long_line = length > self.params['line_length_threshold']
is_thin = (thickness >= self.params['min_line_thickness'] and
           thickness <= self.params['max_line_thickness'])
is_tiny = area < self.params['min_component_area']

# Only remove if it matches ALL noise line criteria OR is tiny
if is_tiny or (is_very_elongated and is_long_line and is_thin):
    # Remove this component
    cleaned[labels == i] = 0
```

If after this entire noise-removal process we lose more than 70% of these pixels, then we remove noise using a skeleton approach; we did not encounter a case where we actually had to do this, but the code to do it is included in our implementation code.

Segmentation

For this stage of the pipeline, we compare the results of 3 different strategies for character segmentation and select the result which yields the desired number of boxes (5 since there are 5 characters). The 3 strategies we use are:

1. Vertical Projection Profile Analysis (this is the preferred and default)
2. Connected Component Analysis
3. Watershed Algorithm

Vertical projection works by computing the vertical (column) projection, smoothing and normalizing the result, and then finding the positive peaks (character centers) using the SciPy function “find_peaks”. If peaks are detected in the image (which they should be in a valid image, then we find the left and right column boundaries using the first and last peaks. Given this column range we then find the individual vertical bounds for each character by looping through the peaks and using row (horizontal) projection. With the horizontal and vertical bounds for each character then we can define the bounding box upper left corner position (x and y), width, and height. Finally, all the boxes are filtered by both minimum width and height. If no suitable peaks are found, then a similar process occurs using the valleys instead.

If vertical projection does not produce suitable results, then connected component analysis is used instead. Similar to the noise removal step, we used “cv2.connectedComponentsWithStats” to make the boxes instead. For the box corner and dimensions, we obtain those from the statistics generated by the OpenCV function and then filter the resulting boxes by size since we have the minimum box width and height requirements. The other backup method is to segment by watershed which uses the watershed algorithm to determine the peaks and valleys of the image similar to the vertical projection approach but with much more complex contour geometry. This approach is like generating a topological map of the image and thus produces more continuous boundaries [8]. We do not prioritize using the watershed algorithm because it requires fine tuning of the markers positions to be most effective and using vertical projection is far simpler while producing reasonable results. Regardless of which of the 3 segmentation methods is

selected, the end result should be bounding boxes which are ready for the final step of post-processing.

Post-Processing

After forming the bounding boxes, post-processing is crucial to make sure that the boxes returned are not empty, overlapping excessively, too large, or too great in quantity. First empty boxes are removed by calculating their area and if it falls below a threshold value (in our case 50), then it is not included in the final list of boxes. Next overlapping boxes are removed by computing the intersection over union (IOU) for the overlapping boxes and comparing that metric to a threshold value. The IOU is calculated by finding the area of the overlap (intersection) and dividing that by the combined area of the two boxes (union) [7]. If the IOU for the two boxes is greater than the threshold we have set (which means the boxes have sufficient overlap), then the boxes are merged using the following logic:

```
iou = self.compute_iou(current, box2)
if iou > self.params['merge_iou_threshold']:
    # Merge boxes
    x1 = min(current['x'], box2['x'])
    y1 = min(current['y'], box2['y'])
    x2 = max(current['x'] + current['width'], box2['x'] +
box2['width'])
    y2 = max(current['y'] + current['height'], box2['y'] +
box2['height'])

    current = {'x': x1, 'y': y1, 'width': x2 - x1, 'height': y2 - y1}
```

After merging overlapping boxes, we can split boxes that are too wide since they likely contain more than one character. To do this we just compare the box's width to the expected width (of a character) times 1.6 (to allow for some variation). If the box is wider than this threshold, then we calculate the number of characters that are likely in the box using that same expected width parameter and divide the box into that number of segments. For example, if a box is wide enough to contain 3 characters, then it is split into 3 smaller boxes. Next we adjust the box count to ensure that exactly 5 boxes are returned. If there are too few boxes, we split the widest ones, and if there are too many boxes then we remove the smallest boxes. Finally after ensuring that there are exactly 5 boxes, then we sort the boxes from left to right and clamp the boxes to the image boundaries.

How is the Image Processing Pipeline Used

With the completion of the bounding box post-processing, that completes the baseline image processing pipeline. This “batch_prelabel” function can be used to run this pipeline on an entire batch of CAPTCHA images and save the bounding box annotations to a JSON output file. However, the primary way we use the pipeline is with the next class “ParameterTuner”. This Python class is used to perform systematic hyperparameter tuning for the CAPTCHA segmentation pipeline using an annotated CAPTCHA dataset.

Annotated CAPTCHA Dataset

Since the objective of this implementation is to test how good of a pipeline we could get by automatically optimizing the parameters, we needed a dataset to test the pipeline variations on. For this dataset, we manually annotated 60 CAPTCHA images from the Kaggle dataset with bounding boxes using a web browser based tool which allowed us to export the bounding boxes and their associated image filename to a JSON file. Each image is annotated with the 5 approximate bounding boxes around each character determined by our best judgement and accuracy we could obtain with a cursor.

ParameterTuner

The ParameterTuner class is initialized with an annotated dataset and the path to the images folder. From there it sets the search space for all the various parameters that we want to optimize. These are the bounds that we tested with:

```
self.param_bounds = {  
    # Preprocessing  
    'clahe_clip_limit': (1.5, 4.5),  
    'bilateral_d': (5, 15),  
    'gaussian_kernel': (3, 7), # Must be odd  
  
    # Thresholding  
    'adaptive_block_size': (7, 21), # Must be odd  
    'adaptive_C': (0, 6),  
    'otsu_weight': (0.3, 0.7),  
  
    # Morphology  
    'morph_kernel_size': (1, 4),  
    'closing_iterations': (1, 3),  
  
    # Noise removal
```

```

    'min_component_area': (10, 40),
    'aspect_ratio_threshold': (7.0, 15.0),
    'line_length_threshold': (100, 150),

    # Segmentation
    'projection_sigma': (1.0, 4.0),
    'min_valley_prominence': (0.05, 0.25),
    'expected_char_width': (15, 35),

    # Post-processing
    'merge_iou_threshold': (0.2, 0.5),
    'min_box_width': (6, 15),
    'min_box_height': (12, 25),
}

```

After initialization, there are two options: either 1) “grid_search_coarse” which is a quick way to search a large region of parameters and find smaller pockets within that range to do more extensive testing, or 2) “optimize_with_cross_validation” which performs a more thorough parameter optimization which avoiding overfitting. We primarily used the latter approach due to our time constraints and desire to get the best results in the time we had. The coarse search could be used on more difficult cases or to improve our pipelines further on a much larger region of parameters.

“optimize_with_cross_validation” performs a K-fold optimization using the SciKit learn function. K-fold optimization works by first splitting the training data (images and their associated annotations) into k equally sized “folds” or subsets (our default is 5), then performing differential evolution (using the SciPy function of the same name) on each of these folds for “max_iterations // n_folds” iterations per fold. The differential evolution process is trying to optimize the following features:

```

# Composite objective: weighted combination of metrics
mean_iou = np.mean([m['mean_iou'] for m in all_metrics])
detection_rate = np.mean([m['detection_rate'] for m in all_metrics])
box_count_error = np.mean([m['box_count_error'] for m in all_metrics])

# We want to maximize IoU and detection rate, minimize box count error
# Return negative for minimization
score = -(0.6 * mean_iou + 0.3 * detection_rate - 0.1 * box_count_error)

```

“mean_iou” has the greatest weight because we are trying to optimize our model to produce the results of our manual labelling of all the images, thus we give it the most incentive to do so. After differential evolution is completed with optimizing the parameters for that fold, the best parameters and associated training score are extracted from the result. With the resulting parameters, the pipeline is then run once more on the validation(test) set (a group of images from a random fold that is preselected when the images are initially split into folds). After each fold goes through this process, the best fold is selected from the results based on its validation score and the parameters from this fold are returned to be saved to a JSON file.

SegmentationDiagnostics - how well is a pipeline performing?

This class is a comprehensive function to automatically score how well a particular pipeline is performing. We primarily use the “batch_diagnostic_report” function in this class because it gives us the overall performance of the model. For more finer details, there are options to just process a single image. When generating a report, the function creates 4 different buckets to sort images into based on the IOU score of the particular image. For each image in the provided directory, the function loops through processing the image through the pipeline, saving the resulting bounding boxes, and computing the mean IOU score for the 5 bounding boxes per image. The images are then sorted via the following logic:

```
if mean_iou > 0.7:
    excellent.append(entry)
elif mean_iou > 0.5:
    good.append(entry)
elif mean_iou > 0.3:
    poor.append(entry)
else:
    failed.append(entry)
```

The resulting information is then returned to the user in a report like the following:

DIAGNOSTIC REPORT

=====

Dataset: 60 images

Performance Distribution:

Excellent (IoU > 0.7): 2 (3.3%)
Good (IoU 0.5-0.7): 8 (13.3%)
Poor (IoU 0.3-0.5): 49 (81.7%)
Failed (IoU < 0.3): 1 (1.7%)

Mean IoU: 0.427

Box Count Distribution:

Correct (5 boxes): 60 images
Under-segmented (<5): 0 images
Over-segmented (>5): 0 images

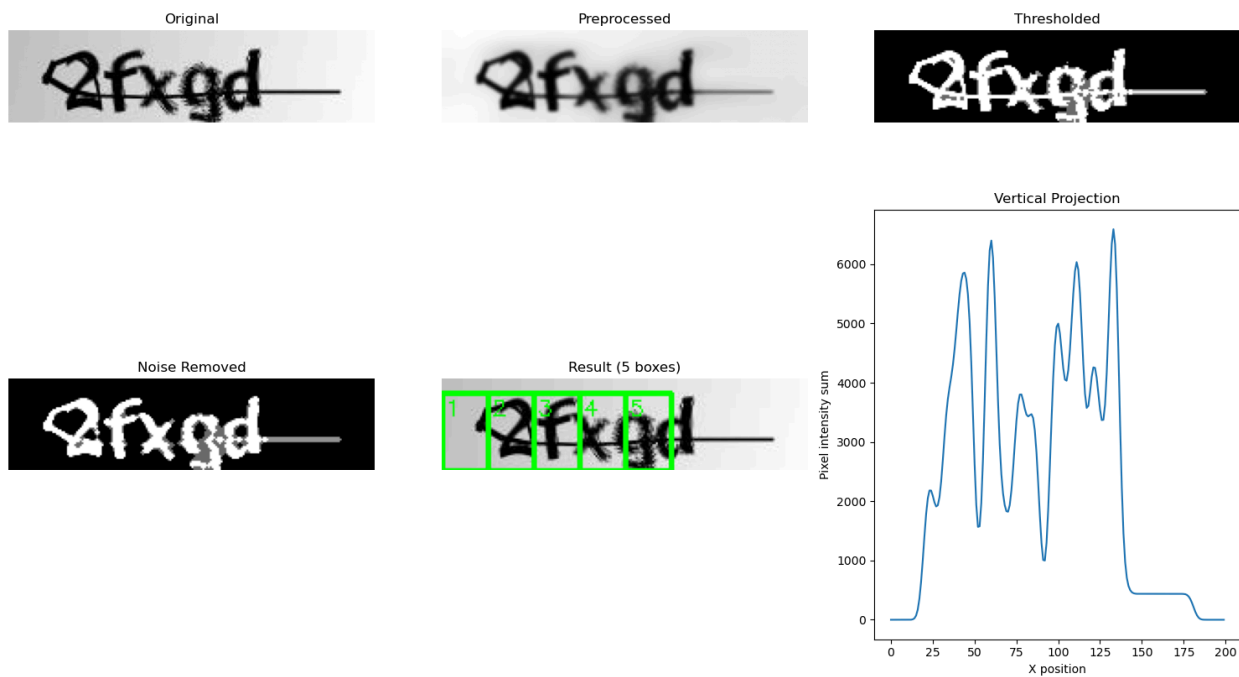
=====

FAILED CASES (IoU < 0.3):

2cgyx.png: IoU=0.297, Boxes=5/5

Based on these statistics, the user can understand how well their current pipeline is working and evaluate how their changes are affecting the pipeline's performance.

Diagram of Pipeline Steps



2. Implementation Analysis

2.1 Approach 1: Projection Profile Segmentation

Overall the vertical and horizontal projection strategy was not completely accurate. The majority of images did not have the 5 correct bounding boxes. In fact, most images are undersegmented, most likely due to our final bounding box filtering step. Moreover, through visual inspection of initial results, our approach seemed inconsistent. However, visual inspection is not sufficient. To actually interpret our results and compare the two approaches against each other, we decided to implement an Intersection Over Union (IOU) approach. In this, we decided to manually place bounding boxes on the 60 images of our dataset. This created a “ground truth” for our approaches to then compare themselves to. In doing so, IOU allows us to get a number that quantifies the degree of overlap between the ground truth and model prediction bounding boxes. In general, an IOU of 1.0 signifies perfect overlap while an IOU = 0 signifies no overlap. An IOU > 0.5 is generally considered good while an IOU < 0.3 is considered a poor match. This accuracy metric penalizes bounding boxes that are shifted left/right or top/bottom and those that are too wide, too narrow, or incorrectly shaped. Therefore, we use IOU to measure “how correct” our predicted character bounding boxes were.

Using the ground truth bounding boxes for the 60 dataset images, we calculated the IOU for the vertical and horizontal projection approach. We also filtered the top 3 results and bottom 3 results by IOU score. Below are the results

```
=== SEGMENTATION SUMMARY ===
Images evaluated : 60
Mean IoU       : 0.538
Median IoU     : 0.542
Min / Max IoU  : 0.356 / 0.750

Top 3 Best:
  34fxm.png: IoU=0.750, Pred=4, GT=5
  2nx38.png: IoU=0.691, Pred=4, GT=5
  2enf4.png: IoU=0.690, Pred=4, GT=5

Worst 3:
  2x7bm.png: IoU=0.374, Pred=2, GT=5
  32dnn.png: IoU=0.361, Pred=2, GT=5
  22d5n.png: IoU=0.356, Pred=2, GT=5
```

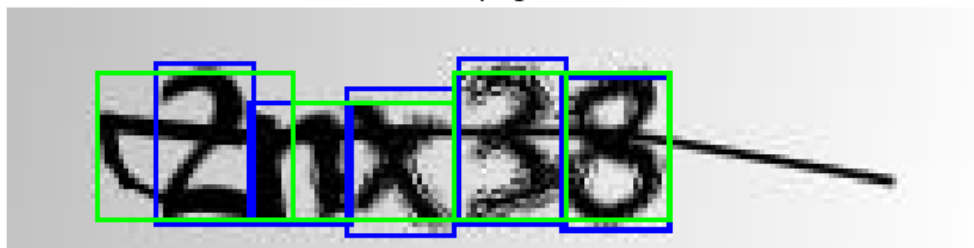
We can see in the figure how on average (mean IOU) the predicted bounding boxes overlapped about 54% with the ground truth box. This indicates partial correctness: the algorithm consistently found roughly the right region, but placement or width/height is off. The median being similar to the mean, signifies that performance was relatively stable across images. There was no extreme skew where a few good results pulled up the average.

On the best performing images, the segmentation is quite close to the correct bounding boxes (75% overlap). This suggests the approach is capable of high accuracy when characters are well-separated or noise is minimal. On the worst performing images, the approach overlaps only 35% with the ground truth. These are likely cases where touching characters, the strokewidth line, or noisy boundaries cause the projection-based cuts to fail. Here are some examples of images which our pipelines performed the best on, where blue is ground-truth and green are the generated boxes:

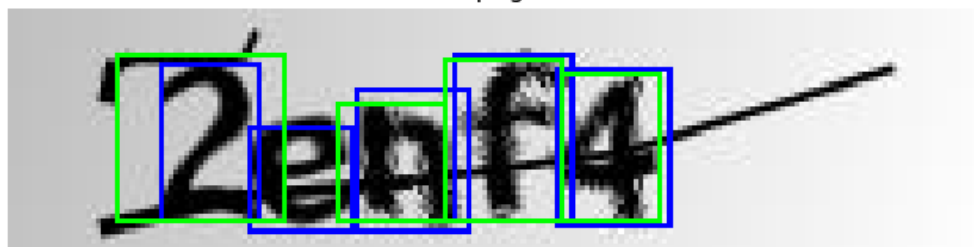
BEST: 34fxm.png IoU=0.750



BEST: 2nx38.png IoU=0.691

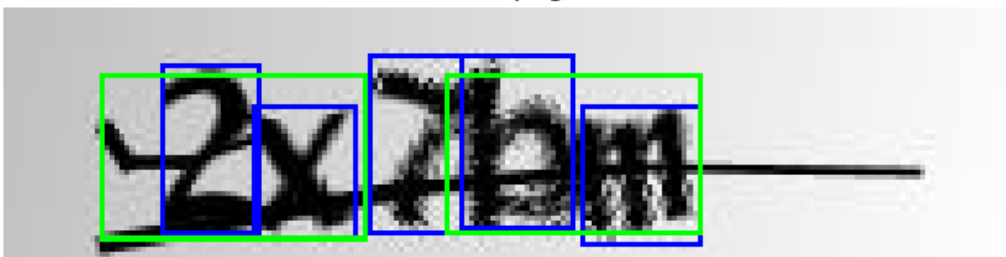


BEST: 2enf4.png IoU=0.690

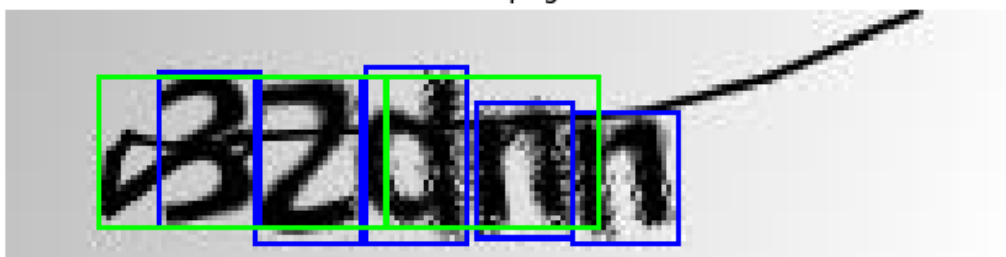


We can see from the best performing images that even these have incorrect bounding boxes. For example in image in image “34fxm”, the 4 bounding boxes produced are almost perfectly correct. However, here it is missing the bounding box on the “4”. Even though predicted character counts are still low (Pred=4 instead of 5), the boxes that were produced aligned well with the ground truth boxes. Similarly, in the other two images we can see how the bounding boxes can overlap with two characters at the same time (“2” and “n” in “2nx38”). This seems to be a common cause of error: touching characters. Missing one character is common when valleys between two letters are too shallow. Also the bounding boxes also tend to include the strokewidth line, another cause of error. Overall, these results suggest that the approach works very well when characters do not overlap heavily and when the smoothing + Otsu pipeline creates a clean vertical profile. Now we can take a look at some of the worst performing images:

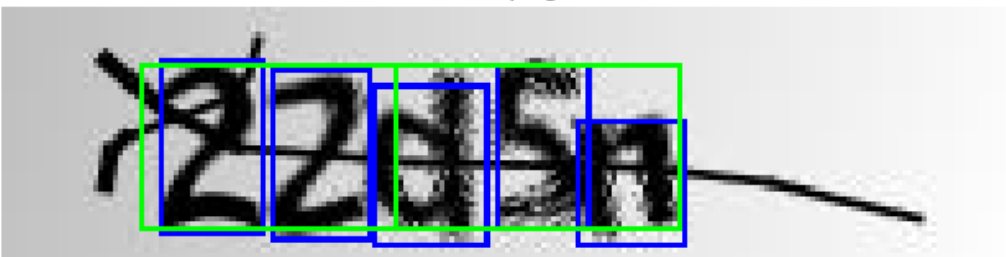
WORST: 2x7bm.png IoU=0.374



WORST: 32dnn.png IoU=0.361



WORST: 22d5n.png IoU=0.356



On the worst performing images we can see how the same issues present themselves just in a worse way. Here the bounding boxes in all three images take up

multiple characters and even include the strokewidth line. In all three images, we can see how only two bounding boxes are created instead of the correct 5. These results could be due to a multitude of issues. Again, this is caused by the characters touching or overlapping which causes the projection profile to not find any clear valleys. The strokewidth line dominates the vertical profile which causes the false peaks/minimas. The smearing or internal character holes can cause Otsu segmentation to become inconsistent. However, one main potential cause of error could be the parameter tuning we did. Since this was done through visual inspection on a handful of images, the parameters were only tuned to those examples. The hyperparameters which cause optimal results for those images during testing do not necessarily translate perfectly to all the dataset images. Our hyperparameter tuning process is not perfect and resulted in a compromise. This would naturally result in certain images completely failing to produce accurate bounding boxes.

Overall, the system detects the general text region reliably. However, as we expected in our proposal, the strokewidth line and spacing between characters caused the most issues. As seen in the images above, the approach struggles with precise individual character separation, especially in noisy or tightly-packed CAPTCHAs. The strokewidth line only adds a layer of complexity. In some images our processing steps were unable to remove the line. Thus, the resulting bounding boxes mistakenly included portions of the strokewidth line in the boxes. Moreover, this is one of the main challenges that arose during the implementation and testing of this approach. We were not able to completely remove the strokewidth line. The pixel width of the line itself is meant to be very similar (sometimes exactly similar) to the character width. Its placement also sometimes crosses through character holes or the characters themselves. The variability of this placement is what makes each image distinct and hard to process. We did not realize how hard it is to create a viable approach with tuned hyperparameters that works on all images without fail.

2.2 Approach 2: Parameter Auto-Tuning using IOU

Our second approach at base level ended up using a very similar approach to the first approach in that the baseline image processing steps are all very similar. The truly remarkable thing to us was that the automated parameter tuning actually worked and did not take hours and hours of searching like a manual approach would have taken. Yes, writing the code did take a long time, but these hours of debugging helped us to stray away from the often arbitrary process of looking at images tuned with different parameters and choosing the best one without entirely understanding how that one parameter can affect the results of all the other parameters. Clearly, we have not maximized the potential of this tool, so there is definitely a lot more time that can be invested in fully understanding and optimizing the optimization tool, but our initial results prove that is in-fact a viable solution and actually did not take too long to complete. These are some of the results from the searching process which was completed in around 5 minutes:

Loaded 60 annotated images

Running optimization with 5-fold cross-validation...

Maximum iterations: 50

Fold 1/5

Training on 48 images, validating on 12 images

differential_evolution step 1: $f(x) = -0.2840825793158136$

differential_evolution step 2: $f(x) = -0.2840825793158136$

differential_evolution step 3: $f(x) = -0.2840825793158136$

differential_evolution step 4: $f(x) = -0.2840825793158136$

differential_evolution step 5: $f(x) = -0.28877385899225516$

differential_evolution step 6: $f(x) = -0.28877385899225516$

differential_evolution step 7: $f(x) = -0.29697681393984093$

differential_evolution step 8: $f(x) = -0.2998956733468068$

differential_evolution step 9: $f(x) = -0.31355533676141256$

differential_evolution step 10: $f(x) = -0.3228177944982773$

Polishing solution with 'L-BFGS-B'

Training score: 0.3228

Validation score: 0.2619

Fold 2/5

Training on 48 images, validating on 12 images

differential_evolution step 1: $f(x) = -0.28880624563000473$

differential_evolution step 2: $f(x) = -0.28911112209325524$

differential_evolution step 3: $f(x) = -0.29350224633630667$
differential_evolution step 4: $f(x) = -0.29350224633630667$
differential_evolution step 5: $f(x) = -0.3104729459948663$
differential_evolution step 6: $f(x) = -0.3104729459948663$
differential_evolution step 7: $f(x) = -0.3104729459948663$
differential_evolution step 8: $f(x) = -0.3104729459948663$
differential_evolution step 9: $f(x) = -0.3104729459948663$
differential_evolution step 10: $f(x) = -0.3104729459948663$
Polishing solution with 'L-BFGS-B'

Training score: 0.3105

Validation score: 0.2944

Fold 3/5

Training on 48 images, validating on 12 images

differential_evolution step 1: $f(x) = -0.28905908794310187$
differential_evolution step 2: $f(x) = -0.28905908794310187$
differential_evolution step 3: $f(x) = -0.28905908794310187$
differential_evolution step 4: $f(x) = -0.28905908794310187$
differential_evolution step 5: $f(x) = -0.28905908794310187$
differential_evolution step 6: $f(x) = -0.28905908794310187$
differential_evolution step 7: $f(x) = -0.28905908794310187$
differential_evolution step 8: $f(x) = -0.28905908794310187$
differential_evolution step 9: $f(x) = -0.28905908794310187$
differential_evolution step 10: $f(x) = -0.2893333961096346$
Polishing solution with 'L-BFGS-B'

Training score: 0.2893

Validation score: 0.3744

Fold 4/5

Training on 48 images, validating on 12 images

differential_evolution step 1: $f(x) = -0.27960610186604773$
differential_evolution step 2: $f(x) = -0.28649807365461244$
differential_evolution step 3: $f(x) = -0.2917276311497272$
differential_evolution step 4: $f(x) = -0.2917276311497272$
differential_evolution step 5: $f(x) = -0.2983242192048253$
differential_evolution step 6: $f(x) = -0.3105129500475169$
differential_evolution step 7: $f(x) = -0.3136055663695182$
differential_evolution step 8: $f(x) = -0.32314878013868215$
differential_evolution step 9: $f(x) = -0.34347165957504683$
differential_evolution step 10: $f(x) = -0.34347165957504683$
Polishing solution with 'L-BFGS-B'

Training score: 0.3435

Validation score: 0.2319

Fold 5/5

Training on 48 images, validating on 12 images

differential_evolution step 1: $f(x) = -0.28216999513447943$

differential_evolution step 2: $f(x) = -0.2868036709932109$

differential_evolution step 3: $f(x) = -0.3019253649507321$

differential_evolution step 4: $f(x) = -0.3019253649507321$

differential_evolution step 5: $f(x) = -0.3019253649507321$

differential_evolution step 6: $f(x) = -0.3019253649507321$

differential_evolution step 7: $f(x) = -0.3023363629294116$

differential_evolution step 8: $f(x) = -0.3023363629294116$

differential_evolution step 9: $f(x) = -0.3023363629294116$

differential_evolution step 10: $f(x) = -0.3185747362967921$

Polishing solution with 'L-BFGS-B'

Training score: 0.3186

Validation score: 0.2814

=====

CROSS-VALIDATION RESULTS

=====

Average validation score: 0.2888

Best fold: 3 (val score: 0.3744)

Best parameters:

clahe_clip_limit: 1.561100784489852

bilateral_d: 15

gaussian_kernel: 3

adaptive_block_size: 7

adaptive_C: 4.480564314015752

otsu_weight: 0.4283164592251554

morph_kernel_size: 3

closing_iterations: 1

min_component_area: 34

aspect_ratio_threshold: 15.0

line_length_threshold: 125

projection_sigma: 2.2145879418983436

min_valley_prominence: 0.2496091995243174

expected_char_width: 26

merge_iou_threshold: 0.0

min_box_width: 11

min_box_height: 13

From these results you can see clear improvements to the pipeline with each iteration of the differential evolution algorithm. These are the accompanying diagnostics from that model and some samples of the results:

DIAGNOSTIC REPORT

=====

Dataset: 60 images

Performance Distribution:

Excellent (IoU > 0.7): 2 (3.3%)

Good (IoU 0.5-0.7): 8 (13.3%)

Poor (IoU 0.3-0.5): 49 (81.7%)

Failed (IoU < 0.3): 1 (1.7%)

Mean IoU: 0.427

Box Count Distribution:

Correct (5 boxes): 60 images

Under-segmented (<5): 0 images

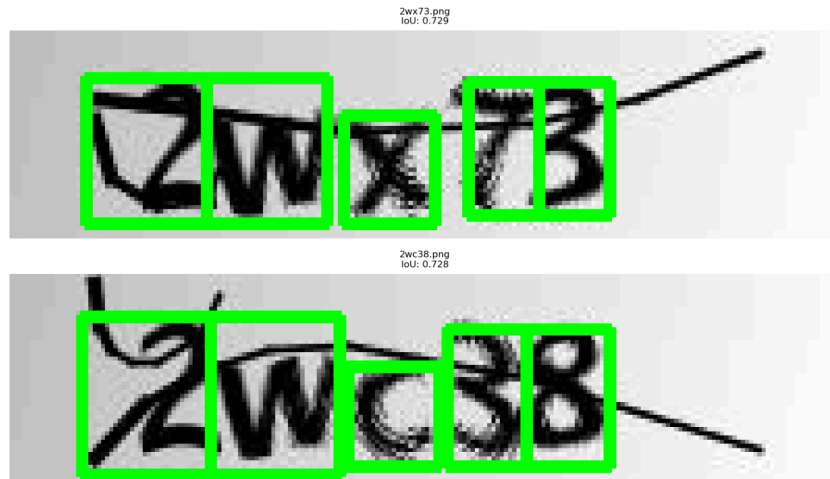
Over-segmented (>5): 0 images

=====

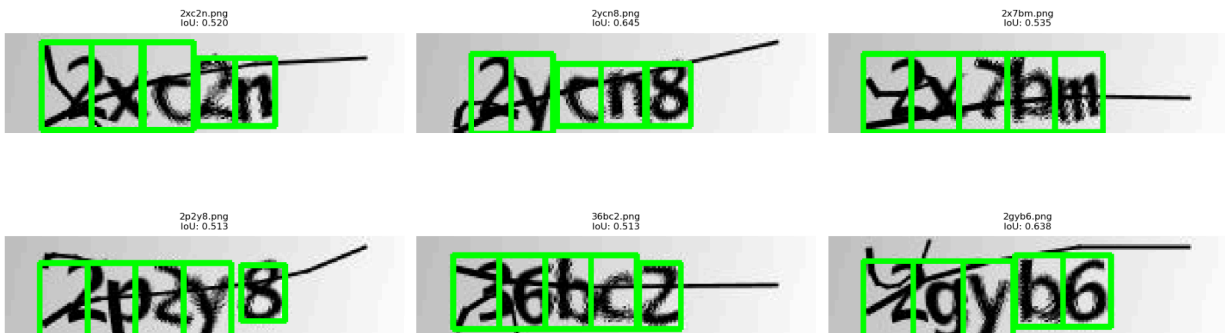
FAILED CASES (IoU < 0.3):

2cgyx.png: IoU=0.297, Boxes=5/5

Excellent Results (IOU > 0.7)



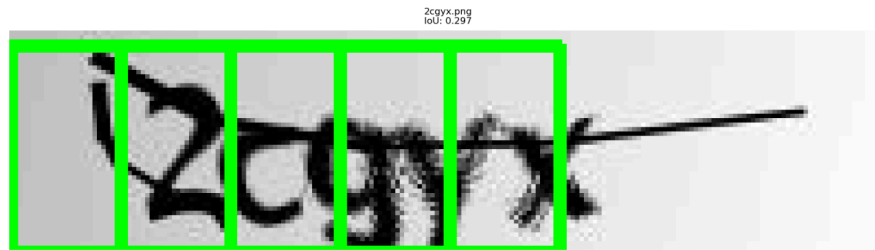
Good Results (IOU 0.5-0.7)



Poor Results (IOU 0.3-0.5)



Failed Result (IOU < 0.3)



From these results we can definitely see a lot of positive signs compared to other implementations we previously worked on for this project. Are the results as good as the manually tuned approach (approach 1)? No, and that is acceptable for now. There were a lot of more components to this approach giving the user a large (almost overwhelming) suite of knobs to turn in search of better results, but by specifying some initial parameter ranges, a fairly decent result is possible pretty quickly and that would be difficult to match with the manual approach. This time difference is especially important for more difficult image processing tasks where even getting initial parameters can be a great challenge. Being able to even partially automate that process is a huge accomplishment.

Approach 2 bleeds the edge between conventional image processing and machine learning techniques, and even with these advancements, it still faces some of the same challenges as approach 1. Looking at the poor and failed images, characters are usually pretty close to correctly boxed, but the noise line “tricks” model and this skews all of the boxes over. To fix this issue, it might be as simple as playing with the post-processing function and changing the parameters for when a box is discarded to avoid cases like this. Maybe we only discard boxes when they are less full than the others. Additionally, this shows that we still have room to improve on our noise reduction processes to combat these difficult noise lines. An approach that could help with noise removal is watershed segmentation, as discussed in the implementation details for approach 2, but we did not get around to fully vetting this process out.

3. Proposed Improvements

Overall, approach 1 was more successful than approach 2 from a performance perspective in this round of testing, but long term, approach 2 is probably easier to improve and can most likely exceed the performance of approach 1 with enough time investment. Something both approaches can potentially benefit from is watershed segmentation which is more advanced than the vertical projection technique we used in

both of our approaches. Watershed segmentation produces what is essentially a topographical map of the image because of this is able to produce more continuous contours, which is useful for things like characters. With modern implementations of the watershed segmentation algorithm, you can choose where markers are placed and where they are not, which is useful for filtering things like the noise lines out. Additional improvements that we could make with approach 2 are searching through a larger range of parameters, running more iterations for each fold, doing coarse searches through various parameter ranges to better narrow down the hyperparameter neighborhoods, and labelling more images to have more images per fold or potentially more folds. Another optimization we could make is playing with the conditions for the post processing steps which dictate which boxes we keep and which we get rid of. Especially in approach 2, the leftmost boxes in many of the results just captured noise instead of the actual characters, showing room for improvement. The last improvement we can think of to make our results better is implementing deep learning for post-processing.

References

- [1] fournierp. (2018). *CAPTCHA Version 2 Images*. Kaggle. Retrieved [12/9/2025], from <https://www.kaggle.com/datasets/fournierp/captcha-version-2-images/data>
- [2] https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html
- [3] <https://www.sciencedirect.com/topics/computer-science/bilinear-interpolation>
- [4] https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html
- [5] https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html
- [6] https://docs.opencv.org/3.4/d4/d76/tutorial_js_morphological_ops.html
- [7] <https://www.v7labs.com/blog/intersection-over-union-guide>
- [8] https://docs.opencv.org/4.x/d3/db4/tutorial_py_watershed.html